
Software for Real Time Control

An Open-Source Summary

Thomas Debelle



April 20, 2026

Contents

1	Introduction	2
1.1	Design procedure	2
1.2	Developing for embedded: Challenges	2
2	Multitasking	3
2.1	Memory Models	3
2.1.1	Stack	3
2.1.2	Heap	3
2.2	Interrupts	4
2.2.1	Programmable Interrupt Timers - PIT	4
2.2.2	As a form of multitasking	4
2.2.3	Case study: nRF52840	6
2.2.4	Good Practices in Interrupt Handling	6
2.3	Multitasking	7
2.3.1	Threads	7
2.3.2	Role of the scheduler	8
2.3.3	Mutual Exclusion - MUTEX	8
2.3.4	Semaphore	8
2.3.5	Memory Protection Unit	8
2.3.6	Inter Process Communication (IPC)	9
3	Scheduling	9
3.1	Minimalist Multitasking OS	9
3.1.1	Tiny OS	9
3.1.2	Contiki	11
3.2	Scheduling Fundamentals	11
3.2.1	Design time - Static	12
3.2.2	Partially static	12
3.2.3	Dynamic and Preemptive scheduling	12
3.2.4	Scheduler structure	12
3.2.5	Rate Monotonic (RM) Scheduling	13
3.2.6	Earliest Deadline First	14
3.3	Challenges	14
3.3.1	Priority Inversion	14
4	License	15
4.1	Modification	15
4.2	Take down	15

1 Introduction

Definition:

A system wherein being correct depends upon giving the correct answer within a strict deadline

From this, we can differentiate 2 types of deadlines:

- Hard: If we miss the deadline = complete failure
- Soft: If we miss the deadline = degrades performance aka expensive task to fix the error

Interfacing with real life is harder than a deterministic clocked circuits. Here, we are facing stochastic event and can be synchronous or asynchronous.

1.1 Design procedure

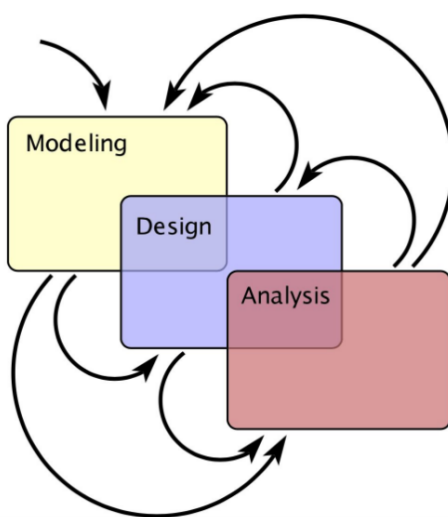


Figure 1: Modeling

We iterate over the design; we try to have those 3 boxes to overlap as much as possible to model and understand the environment we are in. It's an iterative process

- **Modeling / What** are you measuring and controlling?
- **Design / How** will your system achieve your cyber-physical goals?
- **Analysis / Why** does your implementation exhibit this level of correctness, speed, power consumption, etc

1.2 Developing for embedded: Challenges

The main bottleneck is energy:

- Bringing it: cables can be more expensive and less flexible than wireless IC
- Battery doesn't shrink as IC

We also need to make sure our IoT devices can be manageable and avoid spending all the costs and labor on battery management and replacement. Batteries are also toxic and corrosive.

The energy management problem:

- Energy consumption: low power sub-routine
- Energy Storage: alkaline or lipo battery ?
- Energy Production: plug socket ?

2 Multitasking

2.1 Memory Models

Quick recap of memory models in C

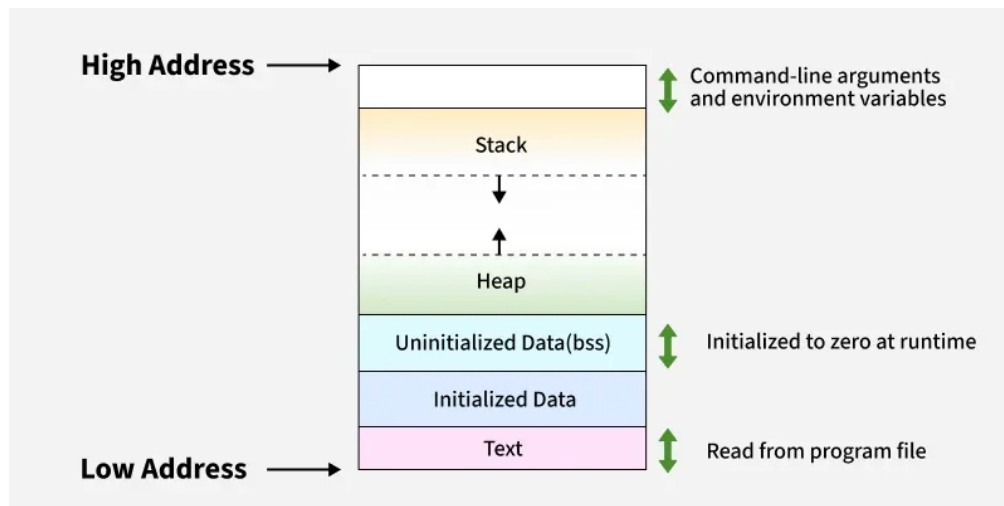


Figure 2: Memory models from GeeksforGeeks

2.1.1 Stack

This is a LIFO datastructure where we push data on it:

- `push`
- `pop`
- `peek`

This is primarily used for procedure call. When we do a call in a function we put in the return address, return content, arguments, and instantiate the local variable. After the function finishes, we free the stack by popping the allocated space and finally check the return address to resume the execution flow.

When we call a function, we set the Program Counter (PC) to the address of the procedure code.

2.1.2 Heap

The heap is like a pile of memory, we can allocate everything how we want. We store there the global variables and can allocate with `malloc` area on the heap. The heap is controlled to see which part of the memory is allocated by which application. To ensure security and avoid applications to do unauthorized read.

```

1  int a = 2; // on Heap
2  void foo (int b, int* c) {
3      ...
4      // value on stack
5  }
6
7  int main (void){
8      int d; // stack
9      int* e; // stack - this is a pointer !
10     d = ...; // Assign some value to d
11     e = malloc (sizeof(int)); // Allocate memory for e. on heap
12     *e = ...; // Assign some value to e.
13     foo (d, e); // d is copy while e is passed by reference
14     ...
15 }

```

2.1.2.1 Stack overflow It happens when a developer expects a return value to a pointer to still be valid after the call. If you try to read this return pointer, you may read corrupted data as the stack was popped and reused by another function call possibly. But those errors can be sneaky and not pop up immediately, always be careful with return value. Best practices is to pass a return pointer managed by the caller where the callee is going to write to it since it is allocated on the heap.

2.2 Interrupts

When interacting with the outside world we need interrupt to interrupt the processing and process incoming data from outside (for example a keyboard, timer).

2.2.1 Programmable Interrupt Timers - PIT

Ticks down values of a counter and then raises an interrupt. We set the interrupt time by writing to a register. We base it on the clock source. It can repeat without SW in the loop providing more deterministic time.

2.2.2 As a form of multitasking

To create a realtime software, we need interrupt that can pause the program and run whatever routine is needed to treat this real-world incoming input. This is done through: **Interrupt Service Routine (ISR)** or **interrupt handler**.

The main possible triggers are:

- **HW interrupts:** physical connection going low, high, falling or rising
- **SW interrupts:** write to a Memory-mapped register
- **Exceptions:** internal HW that detects a fault and jump to a well-known memory location.

The most important and crucial thing with interrupt is **hierarchy**! We also have to enable it.

If enable, the MCU scans the **Interrupt Vector (IV)** for a matching **handler/ISR**. You need to have this matching ISR or an exception will be thrown.

There is the possibility that we get an interrupt while being in an ISR. We can have them and the second one will interrupt the first ISR. Watch out for stack overflow !

2.2.2.1 Serving an interrupt Once we get an interrupt, we will disable all interrupts below and at the same priority level. Give importance to the current one. We indicate the PC and status register on the stack and jump to the **ISR**.

2.2.2.2 Handling the interrupt First, save current context by avoiding over-write on currently used register. (So basically save on the stack previous values). Then, execute ISR logic.

Finally, restore previous state (registers) and resume execution.

2.2.2.3 SysTick Example

Info

Use keyword **volatile** to avoid that the compiler optimize your code and force it to re-read the value each time.
(gcc -o0)

```
1 volatile uint timerCount = 0;
2 void countDown(void){
3     if (timerCount != 0){
4         timerCount--;
5     }
6 }
7
8 SysTickPeriodSet(SysCtlClockGet() / 1000); // Set period relative to main clock
9 SysTickIntRegister(&countDown); // Register ISR that was defined as interrupt handler
10 SysTickEnable(); // Enable the SysTick timer
11 SysTickIntEnable(); // Enable the SysTick interrupt
```

Having interrupts result that atomicity is hard to guarantee since now everything can interrupt the flow of your program.

2.2.3 Case study: nRF52840

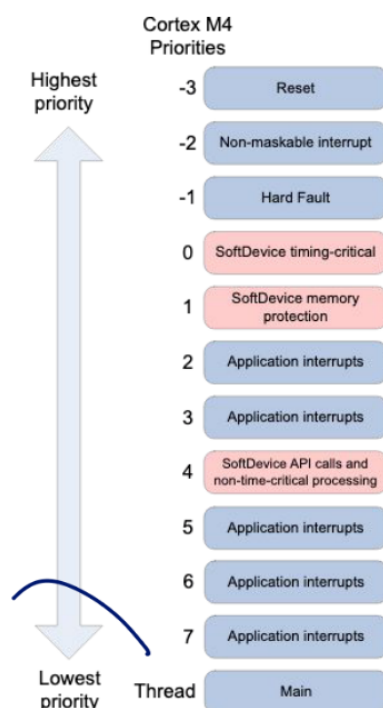


Figure 3: Cortex M4 Priorities

Higher priorities preempt the other interrupts. There are 7 level of interrupts where SoftDevice timing-critical is a “closed source wireless stack that is strongly separated from the application code.”

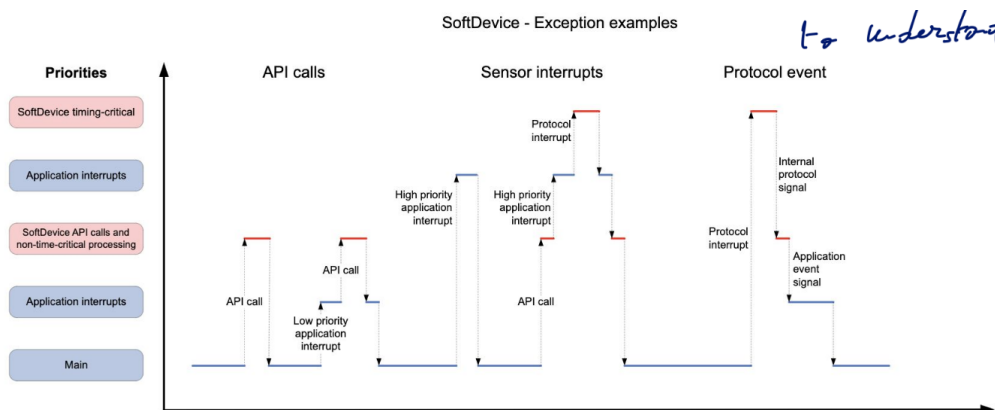


Figure 4: Interrupts visualized

It is important to be able to draw such diagram when creating interrupts. If you can't represent the interrupt flow as depicted, it may be too hard to understand or maintain.

2.2.4 Good Practices in Interrupt Handling

- Prioritize interrupts properly: you cannot do everything! To avoid blocking operations for too long which could result in missed deadlines.

- Keep them short: a main loop for real work, communicate with flags. KISS strategy
- Keep it simple: use state machines.
- Global variables: make sure to declare volatile.
- Using data buffers: be heedful of overflows
- Calling functions in an ISR: discouraged, be cautious. Make the stack more complicated.
- Time critical tasks: understand latency, aim for fixed time interrupts.
- Decide a background/main process: this will simplify your thinking

2.3 Multitasking

We can either do it in pure software or using some hardware mutex.

It is important to understand the fundamental difference between SW and HW. Most SW are declarative as the sequence of code determines the sequence of execution.

This is far from HW where we use FSM and process data concurrently.

2.3.1 Threads

We can have some interleaved execution for threading meaning that execution isn't truly parallel like basic threads make it seem like (context switching).

Moreover, they share the same memory and can access each others' variables (not necessarily any memory protection). Interrupts are kinda like threads since the expected flow of execution can be interleaved with ISR.

2.3.1.1 In an OS Threads are managed by the OS. For example in Linux with the POSIX Pthreads API. The thread start routine can be provided with an argument and can return a value.

- `create`: instantiate and run
- `cancel`: messy stop
- `pause`
- `resume`
- `join`: wait for thread to complete. If we create and don't join, the program will terminate. We have to wait for the threads to complete with this method.

```

1  # include <pthread.h>
2  # include <stdio.h>
3  void* printN (void* arg) {
4      int i;
5      for(i = 0; i < 10; i++) {
6          print f ("My ID: %d\n", *(int*) arg) ;
7      }
8      return NULL;
9  }
10 int main (void){
11     pthread_t threadID1, threadID2;
12     void* exitStatus;
13     int x1 = 1, x2 = 2;
14     pthread_create (&threadID1, NULL, printN, &x1);
15     pthread_create (&threadID2, NULL, printN, &x2);
16     printf("Started threads.\n");

```



```
17 pthread_join(threadlDI, &exitStatus);
18 pthread_join(threadlD2, &exitStatus);
19 return 0;
20 }
```

2.3.2 Role of the scheduler

The most important with threads is the OS scheduler that decides which one to send next. We can use many variant of a scheduler to meet certain goals or improve its performances.

2.3.2.1 Cooperative multi-tasking It's like the 'care bare' version of scheduling. You don't interrupt, you just wait for the thread to yield itself or do an OS call procedure (blocking call).

The scheduler records the stack pointer of the current thread, then modifies it to point to the new thread stack. It pops off the address of the stack and resume execution.

Bad for starvation of other threads in one doesn't yield. We need to proactively yield!

2.3.2.2 Proactive yielding We pair the thread with a jiffy or periodic timer ISR. Basically it is a counter we decrement until it reaches 0 and we interrupt the thread to run another one.

- Smaller jiffy = more interleaving execution but this means we waste time in context switching.
- Larger jiffy = possibly longer blocking statement

2.3.3 Mutual Exclusion - MUTEX

To avoid race conditions we must ensure that writing to a data structure won't be attempted concurrently. We have 1 global variable lock `pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;`. Then we can call `pthread_mutex_lock(&lock);` to lock and block if already in use.

The worse scenario is deadlock where 2 threads could yield each one lock and they both need e

2.3.4 Semaphore

Here it is a counter that indicates how many thread can for example execute at the same time. It is a shared variable. This shared variable (semaphore)'s value restrict the access to other variables.

2.3.5 Memory Protection Unit

Usually, it is never a good thing to let thread access the full memory space (usually we use virtualization for this). For this we need HW support and so we need a MPU or MMU.

Specify various memory regions with different access rules typically. We check if the process has the required access role. Can be used for RAM or Flash.

2.3.5.1 Case study: ARM Cortex MPU 8 memory regions with each 8 sub-regions. We can re-used the MPU to create control access for memory mapped IO peripherals.

- Regions may be updated at runtime.
- Dedicated system exception can be assigned a custom handler.
- Cause of memory faults is stored in special-purpose register.

AP[2:0]	Privileged permissions	Unprivileged permissions	Description
000	No access	No access	All access generate a permission fault
001	RW	No access	Access from privileged software only
010	RW	RO	Writes by unprivileged software generate a permission fault
011	RW	RW	Full access
100	Unpredictable	Unpredictable	Reserved
101	RO	No access	Read by privileged
110	RO	RO	Read only, by privileged or unprivileged software
111	RO	RO	Read only, by privileged or unprivileged software

Figure 5: ARM Cortex MPU

2.3.6 Inter Process Communication (IPC)

- **File-based:** create data to a file and read from it
- **Message passing:** most popular: one process creates a data block, deposits it in shared memory then notifies other processes
- **Network based:** use loopback (network) to send packet-based communication
 - Can extend it for actually remote communication

3 Scheduling

3.1 Minimalist Multitasking OS

3.1.1 Tiny OS

Everything is modular and component-based. We have to explicitly state the services and dependencies. The OS is non-resident SO the OS is compiled with the APP.

Can be made really lightweight with only the required components. Once, compiled, the app is monolithic as everything resides in one binary.

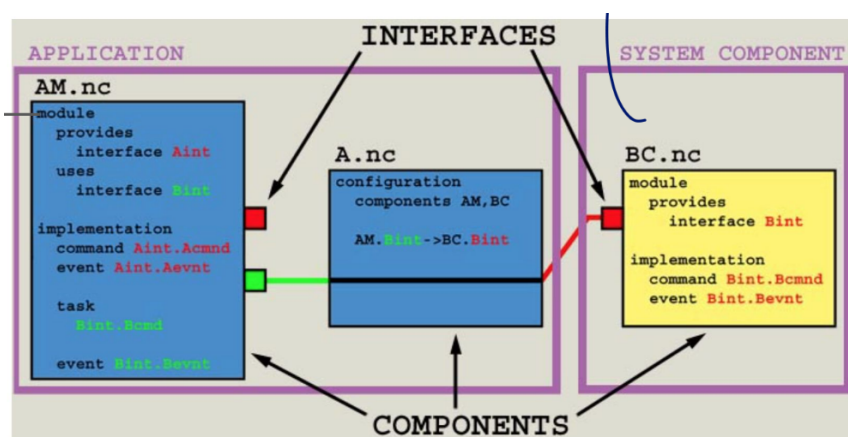
We have to explicitly defined all interfaces and if we want to update the code we have to re-flash it totally.

3.1.1.1 Concurrency model

Table 1: Concurrency model

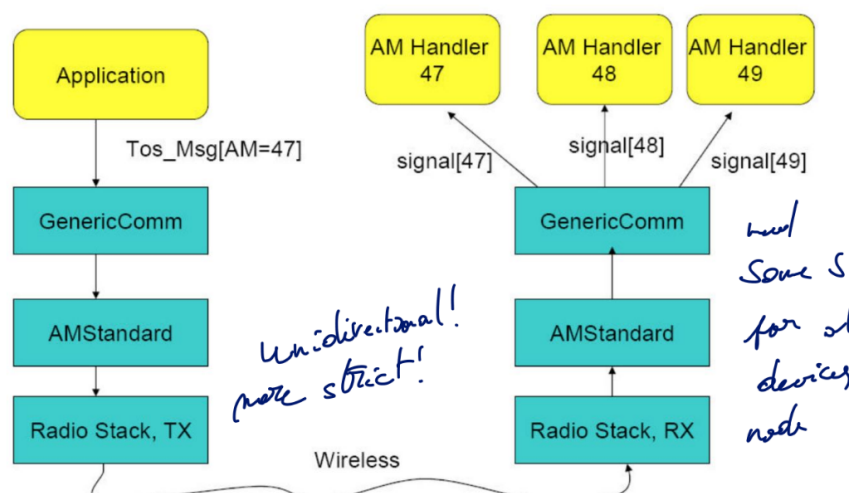
Name	Description
Tasks	compute jobs that may be scheduled or preempted by events
Events	To schedule tasks and pass data. May be generated by OS or by components
Commands	To invoke component functionality

Scheduler is **event based**, when the queue is empty OS goes to sleep. Based on the NesC programming language which is an extension of C.

**Figure 6:** Concurrency model

There is this idea of provided and used interfaces. There are special components “configuration” to indicate how to bind to the interface.

There are also implementation that explains what command is for what event. Task are defined for processing used inputs.

**Figure 7:** Distributed interactions

3.1.1.2 Multitasking

- Multitask without threading so event-based scheduler.
- Isolated memory model
- Process-like encapsulation through components
- Message passing based on IPC

→ Core OS is 400 bytes, target devices had 4kB RAM and 16kB flash.

Problem with it: This can be bad for real time since no guarantee on how long things will take, no priority!

3.1.2 Contiki

3.1.2.1 Concurrency model Also events but uses minimal thread-like unit **proto-thread**. NO stack in the thread! It must be declared statically but easier for developer than dealing with events.

This time the OS is resident with a minimal kernel, drivers, networking. Better for updates and maintenance. The program can be binded statically.

Contiki and Tiny OS both have unrestricted access to device hardware.

It is based on a cooperative scheduler. There is the Contiki API for threads used for yielding them.

3.1.2.2 Modularity of the ELF format Contiki has no distinction between kernel space and user space! All Inter-process communication must occur via events, so go through the kernel. Compiled in the ELF for the linkable format for loading the program but leads to a code size overhead of 50%. The advantage is the possibility for modular loading and on-device linking (which requires those extra meta-data).

This allows for more efficient updates through the ELF format verbose. The files are relocated in the Flash.

So we have:

- Simple threading
- No memory isolation nor process encapsulation
- Message passing IPC
- OS is resident
- → Core OS is 3.8 kB

3.2 Scheduling Fundamentals

The scheduler must decide what task executes next based upon:

1. **Who:** which processor executes
2. **When:** timing constraints of the task
3. **What order:** to avoid pre-emption, enable hierarchy, ...

Decision can be made at *design, load or run-time*.

3.2.1 Design time - Static

This is part of the program code itself

Precise ordering and control of everything. No need for lock since we know and control the flow of execution (timing for mutual exclusion).

Hard to do since, tasks are data-dependent execution time and hard to predict execution time (DVFS, p-state, ...)

3.2.2 Partially static

We simply map task to a given processor (for example background task on the slowest core, ...) but the scheduling is **dynamic**. So here we need mutex to decide when to schedule. But we do not change the order of tasks based on locks.

3.2.3 Dynamic and Preemptive scheduling

This is part of the embedded OS code

3.2.3.1 Fully dynamic We execute ASAP on any processor

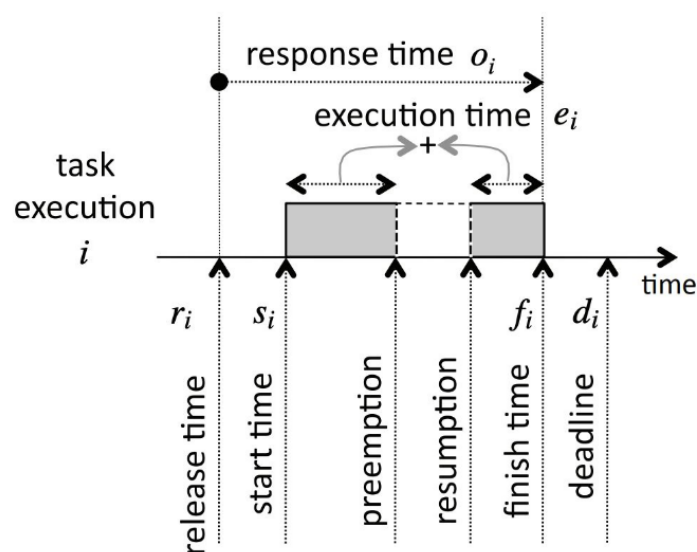
3.2.3.2 Preemptive We can stop a current task to run another instead of waiting for it to become free like in the *fully dynamic*

3.2.4 Scheduler structure

Schedulers operate on Tasks, threads or processes, and based on a scheduling procedure. (contiki: dynamic non-preemptive; tinyOS: static)

Preemption can occur when we acquire or release a mutex, OS API calls (since we need to cross user space and call OS function), Expiration of a timer.

3.2.4.1 Preemption Scheduler records task preemption so that it can restart it later. We set the stack pointer is set to refer to the state of the task to be started or resumed. Finally we can run the other task.



i = Task execution

r^i = Task release/available

s^i = Task start

f^i = Task finish

d^i = Required deadline

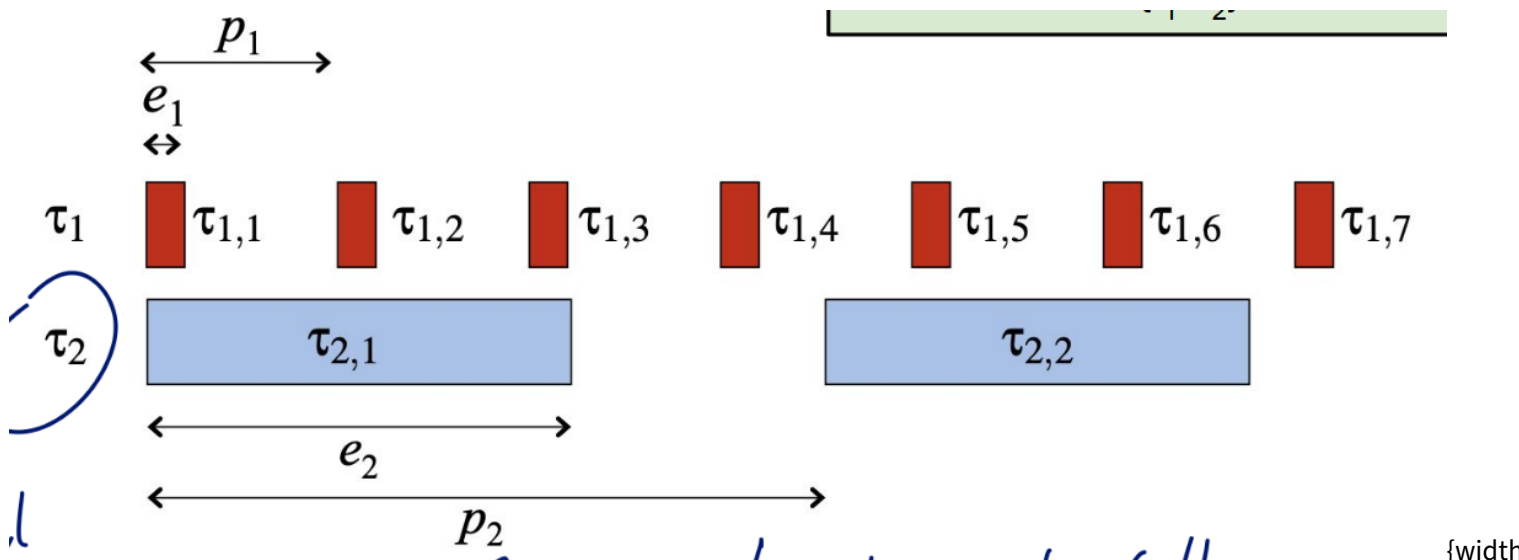
o^i = Response/total time

e^i = Execution time *to releasing time to finish*

60%}

3.2.5 Rate Monotonic (RM) Scheduling

We have t_n Task where each must execute in a period p_i of the task. The deadline for the task i is $r_{i,1} + jp_i$ where $r_{i,1}$ is the release of the task for the first execution. This RM scheduling is **optimal** for this model. Important to note that the period and execution of each task can vary drastically.



60%}

We can see in this example that due to the long execution time of the second task, we will miss some timing deadline for the first one. We put more priority to the first task and can preempt the second one which will make e_2 longer.

The timer for each task is equal to **the greatest common divisor of the periods**. e.g., $p_1 = 5$ and $p_2 = 20$ then timer will be $p = 5$. We won't let any task run longer than the period allowed.

Tip

Make period of the task with some granularity. This allows better insertion of preemption and avoid useless interrupt.

Single task priority !!! Cannot tolerate increase of execution time (cause on the edge so bye bye) or decrease in period.

3.2.6 Earliest Deadline First

This one is assuming **non-repeating** tasks Earliest Due Date. Executes the task with the earliest deadline. Minimize the maximum lateness. Not great for periodic task.

3.2.6.1 Earliest Deadline First Extends EDD with support for periodic tasks and task arrival. If a task is repeated, can be assigned a different priority. For periodic task, the deadline is set to the period of the task. Less preemption = less overhead but more complexity in implementation.

3.3 Challenges

3.3.1 Priority Inversion

Definition

A higher priority task should execute before/have priority on a lower order task. But if this lower order task locks a mutex used by a higher, the higher task could be blocked forcing to yield to a lower order task. Not the desired behavior.

3.3.1.1 Priority Inheritance “When a task blocks on a mutex, the task holding the lock inherits the priority of the blocked task”. So if a lower task blocks a mutex and higher executes and discovers it cannot run, lower will become a higher level and result in a sort of “accelerated execution” of the lower task since it is now more important. Typically, avoid preemption of tasks between the low and high order.

Issue: can deadlock cause of the intertwined priority.

3.3.1.2 Priority Ceiling Protocol Using the right priorities can prevent such deadlock. We assign to each semaphore or lock a priority corresponding to the highest level of priority that can lock it.

“Task t can acquire a lock a only if the task’s priority is strictly higher than the priority ceilings of all locks currently held by other tasks.” So basically, if we lock a lock, it will become of priority equal the highest level that locks this lock. This avoids high priority task to create the aforementioned deadlock situation.

- **OCPP:** Original Ceiling Priority Protocol: boost priority if higher priority task tries to acquire the locked resource.
- **ICPP:** Immediate Ceiling Priority Protocol: task is boosted immediately when attempting to lock. Much better, less context switches (cheaper than priority changes)!

4 License

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0) for KU Leuven students.

You are free to:

- **Share** — copy and redistribute the material in any medium or format with people inside of KU Leuven or being granted access to the source material.
- **Adapt** — remix, transform, and build upon the material if you are a KU Leuven student.

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** — You may not use the material for commercial purposes.
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

For the full legal text of the license, please visit: <https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode>

4.1 Modification

To contribute to this work or any other one from this project please find more information at the Github repository.

4.2 Take down

If you are a professor or representing any official party and wish to take down this work, you can submit a takedown request at this url: https://github.com/Tfloow/ESATSummary/issues/new?template=takedown_notice.yml

Some Rights Reserved 2026 Authors of the Summary, Professors of the Course and possible book's authors. Some Rights Reserved.